cuDESeq2: GPU acceleration of the DESeq2 differential-expression pipeline

Max Highsmith [1,*], [ADDITIONAL AUTHORS TBD] [1]

[1]Synthesize Bio

[*]To whom correspondence should be addressed: max@synthesize.bio

Intended article type: *Original Paper*. Subject: Gene expression.

Abstract

Motivation: DESeq2 is widely used for bulk RNA-seq differential-expression (DE) analysis. Its dispersion and generalized linear model (GLM) fits operate independently by gene, but the reference implementation executes this work on the CPU. Repeated DE analyses can therefore dominate evaluations of gene-expression models.

Results: We present cuDESeq2, a from-scratch PyTorch reimplementation that batches DESeq2's per-gene work on the GPU while reproducing the reference step by step. Against R DESeq2 1.52.0 the closed-form stages agree to floating-point precision and dispersion to 8.5e-4 median relative error, over six real designs (7–300 samples, $P = 2$–5) and 76/76 step-parity tests on synthetic fixtures. The standard Wald pipeline includes `DESeq()`'s count-outlier replacement and per-gene refit. On one A100 cuDESeq2 is 10.3–99.2× faster end-to-end than single-threaded R across the six designs.

Availability and implementation: `gpu-deseq` requires PyTorch and a CUDA GPU, and is open source under the MIT license at `https://github.com/synthesizebio/gpu_deseq`, with reproduction scripts in `bench/` and `benchmarks/`.

Contact: max@synthesize.bio

Supplementary information: Appended to this manuscript.

1 Introduction

DESeq2 (Love et al., 2014) is a standard method for bulk RNA-seq differential-expression analysis. A typical Wald analysis estimates size factors and dispersions, fits a negative-binomial GLM, computes significance, and may shrink log fold changes with apeGLM (Zhu et al., 2019). The dispersion, GLM and shrinkage calculations are iterative and independent by gene. DESeq2 performs these calculations on the CPU; its parallel mode distributes blocks of genes among workers.

Runtime is usually secondary for a single DE analysis. It becomes more important when DE is part of an evaluation loop for gene-expression or virtual-cell models (Bunne et al., 2024; Roohani et al., 2025). Some recent benchmarks use rank-based tests on normalized expression as less expensive surrogates (Roohani et al., 2025; Wei et al., 2026). A faster implementation of the DESeq2 model permits the reference analysis to remain in that loop.

The established route to a faster DESeq2 thus far has been algorithmic and CPU-bound. glmGamPoi (Ahlmann-Eltze and Huber, 2020) fits the Gamma–Poisson GLM 6–13× faster than DESeq2 and edgeR, and DESeq2 can delegate its dispersion and GLM fits to it. PyDESeq2 (Muzellec et al., 2023), a Python reimplementation, improves interoperability and speed on large datasets, though by its authors' own account it yields results "similar, but not identical" to DESeq2; GPU acceleration has meanwhile transformed adjacent single-cell workflows, where rapids-singlecell and ScaleSC report 15–20× end-to-end speedups (Hu et al., 2025), but those gains come from the dense and sparse linear-algebra core: PCA, neighbor graphs, clustering. Those systems accelerate the dense and sparse linear-algebra stages rather than the per-gene negative-binomial inference used for bulk DE. This work targets that inference path.

We implemented cuDESeq2 in PyTorch (Paszke et al., 2019). It batches genes on the GPU and follows R DESeq2 1.52.0 at each intermediate step. Across six real designs, the p95 relative dispersion error is at most 4.4×10^{-3} (Sec. 3.2). The dispersion loop has three execution modes: eager PyTorch, CUDA-graph chunked replay (NVIDIA Corporation,

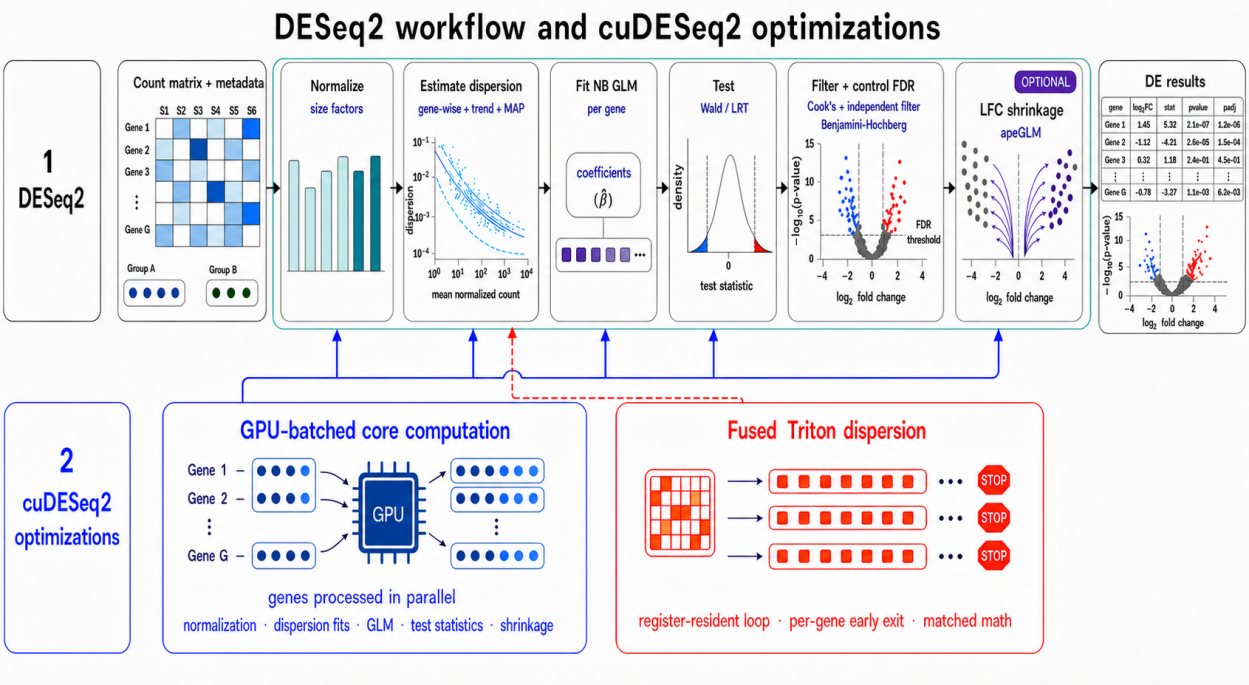

Figure 1: Overview of cuDESeq2. *Top*, the five stages of a DESeq2 run. *Bottom*, where the GPU work goes: DESeq2's per-gene, sequential CPU work (dispersion estimation, the negative-binomial GLM, and apeGLM LFC shrinkage) is batched across all genes (*left*, solid arrows), and the dispersion fit, which is limited mainly by launch latency, has a fully fused Triton implementation (*right*, dashed arrow). A third mode uses CUDA-graph chunked replay; the three modes are compared in Table 1. Every stage is checked against R DESeq2 step by step (Sec. 3.2).

2024), and a fused Triton kernel (Tillet et al., 2019).

Contributions.

(i) A batched GPU port of DESeq2's `fitDisp` (Cox–Reid gradient ascent), operation-for-operation faithful to the reference and validated against R at every intermediate quantity.

(ii) A CUDA-graph mode (chunked replay) that removes launch overhead from the dispersion loops, using a capture-safe no-pivot LU in place of cuSOLVER.

(iii) A fully fused Triton kernel: one program per gene runs the whole gradient-ascent loop in registers with per-gene early exit.

(iv) A batched, on-device port of apeGLM's own L-BFGS optimizer (limited-memory BFGS with a strong-Wolfe line search).

(v) A per-step benchmark against single-threaded R and the three GPU execution modes, with an empirical roofline analysis.

2 Methods

2.1 The DESeq2 pipeline and its cost profile

DESeq2 (Love et al., 2014) models each gene independently with a negative-binomial GLM. For gene g with counts K_{gi} over samples $i = 1, \ldots, S$, design matrix $X \in \mathbb{R}^{S \times P}$, and size factors s_i, it assumes $K_{gi} \sim \mathrm{NB}(\mu_{gi}, \alpha_g)$ with $\mu_{gi} = s_i \exp(x_i^\top \beta_g)$ and variance $\mu_{gi} + \alpha_g \mu_{gi}^2$, and runs five stages. We write S for the sample count and P for the number of model terms throughout; where a formula mirrors R's own source we keep R's names $m \equiv S$ and $p \equiv P$ for it, and dataset sizes are quoted as $n \equiv S$.

1. **Normalization**: median-of-ratios size factors.
2. **Dispersion estimation**: gene-wise Cox–Reid MLE, a parametric mean–dispersion trend, empirical-Bayes MAP shrinkage, and an outlier rule. This is the dominant cost on the largest design tested here.
3. **GLM fit / testing**: IRLS negative-binomial GLM, Wald / LRT.
4. **Significance**: Cook's-distance filter, independent filtering, Benjamini–Hochberg adjustment.
5. **LFC shrinkage**: apeGLM (Zhu et al., 2019), a Cauchy-prior MAP estimate of the effect size.

Stages 2, 3 and 5 contain per-gene iterative optimizations, and each is parallel across genes. On the largest design, dispersion estimation dominates the wall time (Sec. 3.3) because its

gene-wise MLE maximizes the Cox–Reid adjusted profile log-likelihood

$$\ell_{\mathrm{CR}}(\alpha_g) = \sum_{i=1}^{S} \log \mathrm{NB}\big(K_{gi}; \mu_{gi}, \alpha_g\big) \;-\; \tfrac{1}{2} \log \det\big(X^\top W_g X\big), \tag{1}$$

with $W_g = \mathrm{diag}\big(\mu_{gi}/(1+\alpha_g\mu_{gi})\big)$. The log det Cox–Reid adjustment couples every sample and has to be recomputed at each iteration.

2.2 A parity-first batched port

We reproduce the reference computation step by step so that intermediate quantities agree with R (Sec. 2.8). In the eager tensor path, each per-gene loop in R becomes a program over all G genes at once: the per-gene solver state ($\log \alpha_g$, β_g, step sizes, iteration counters, convergence flags) is carried as $(G, \cdot)$ tensors, and a gene that has converged is frozen by a mask so that its later updates are no-ops. Batch speeds are constrained to the slowest gene's iteration count. Genes that are zero in every sample are dropped before fitting and their outputs padded with NaN, as in R.

Three of the five stages need no adjustment beyond this batching. *Normalization* computes median-of-ratios size factors from a 0.5 quantile rather than a median primitive, so that the averaging of the two middle values at even sample counts matches R; when every gene carries at least one zero it falls back to the positive-counts geometric mean. The *GLM fit* is a batched IRLS negative-binomial solve: working weights $W = \mu/(1+\alpha\mu)$, working response $z = \log(\mu/s) + (K-\mu)/\mu$, and normal equations $(X^\top W X + \lambda I)\beta = X^\top W z$ solved for all genes in one batched factorization (ridge $\lambda = 10^{-6}$, deviance-ratio convergence $< 10^{-8}$, at most 250 iterations; a gene passing $|\beta| = 30$ is marked diverged). It returns β, an unthresholded μ, which Cook's distances need raw, and the hat-matrix diagonals; the few genes that reach the cap or diverge are re-fit one at a time by L-BFGS-B on the host, mirroring DESeq2's `optim`-based fallback for the same genes. Wald standard errors follow from the sandwich form $\mathrm{SE} = \sqrt{c^\top (M+\lambda I)^{-1} M (M+\lambda I)^{-1} c}$, with $M = X^\top \tilde{W} X$ built from a μ floored at 0.5

to match `fitBeta.cpp`. Without that floor a sample with $\mu < 0.5$ contributes almost no weight instead of a small one, M effectively loses it, and the SE inflates on degenerate genes (Sec. 3.2.1). The LRT statistic is the full/reduced deviance difference.

Significance contains no per-gene optimizer; Cook's distances and the BH adjustment are closed-form. A gene is flagged when its largest Cook's distance over samples in design cells of at least three replicates exceeds $F_{0.99,P,S-P}$, computed against a trimmed method-of-moments dispersion floored at 0.04, and fewer than three samples carry more counts than the sample holding that maximum. Independent filtering sweeps 50 base-mean quantile cutoffs, runs BH at each, and selects the first whose rejection count comes within one RMS residual of the peak of a robust LOWESS fit to the rejection curve.

The public `deseq()` entry point runs the standard Wald path: normalization, dispersion estimation, the Wald fit, Cook's-distance replacement and refitting for eligible design cells, and result assembly. The lower-level functions remain available for stage-wise validation and for LRT analyses. We use "drop-in" in this computational sense: the standard input model and reported DE quantities are reproduced. It does not imply that the Python API accepts every R object or every optional DESeq2 estimator (Sec. 4.1).

2.3 Batched dispersion estimation

The dispersion stage is a batched port of DESeq2's `fitDisp` and the `estimateDispersions` logic around it. Two inputs must be built the way R builds them, because the fit's accept-and-revert behavior depends on where the optimizer starts: the initial dispersion is $\mathrm{clip}\big(\min(\alpha_{\mathrm{rough}}, \alpha_{\mathrm{MoM}}), 10^{-8}, 10\big)$ from a linear-regression and a method-of-moments estimate, and $\hat{\mu}$ comes from plain linear regression for saturated designs and one IRLS pass at the initial dispersion otherwise. The stage has three parts.

(i) Gene-wise Cox–Reid MLE. We maximize Eq. 1 over $a = \log \alpha$ by gradient ascent with Armijo backtracking. Each iteration proposes $a' = a + \kappa\, \partial_a \ell_{\mathrm{CR}}$, shrinking κ as needed to keep a' inside $[-30, 10]$, accepts when the Armijo condition holds ($\varepsilon = 10^{-4}$), and adapts

the step: on acceptance $\kappa \leftarrow \min(1.1\kappa, \kappa_0)$ with $\kappa_0 = 1$ and a halving every fifth acceptance, on rejection $\kappa \leftarrow \kappa/2$. A gene stops once an accepted step improves the objective by less than 10^{-6} or a falls below $\log(\text{minDisp}/10)$, capped at 100 iterations. Two R-matching safeguards follow: a *noIncrease revert* restores the initial estimate for genes whose objective never improved on it, and a *grid fallback* re-fits the genes R would flag as non-converged (those at the cap or with a single accepted step), whose dispersion still exceeds $10 \cdot \text{minDisp}$, using a deterministic 100-point coarse grid over $\log \alpha$ refined by a second. Estimates are clipped to $[10^{-8}, \max(10, S)]$: R's ceiling is $\max(10, S)$, and a fixed 10 truncates legitimate dispersions above ten samples.

(ii) Parametric trend. We fit $\alpha \approx a_0 + a_1/\bar{\mu}$ over the genes with dispGeneEst $> 100 \cdot \text{minDisp}$ from $(a_0, a_1) = (0.1, 1)$. Each iteration forms residuals $\alpha/(a_0 + a_1/\bar{\mu})$, keeps the genes whose residual lies in $(10^{-4}, 15)$, and re-fits a Gamma-family, identity-link GLM warm-started at the current coefficients, stopping when $\sum \log(\text{coef}/\text{coef}_{\text{old}})^2 < 10^{-6}$ with the inner GLM converged, or after ten iterations; a trimmed-mean trend is the fallback. R's other two `fitType` settings are also implemented: `"local"`, a LOWESS of $\log \alpha$ on $\log \bar{\mu}$ at span 0.3 with three robustness iterations, and `"mean"`, which forces the trimmed-mean trend.

(iii) MAP shrinkage. A log-normal prior centered on the trend shrinks the gene-wise estimates. Its variance follows R's three regimes, each built from the MAD s of the log-residuals: the closed form $\max\big(s^2 - \psi_1(\frac{m-p}{2}), 0.25\big)$ at ample residual degrees of freedom, a stochastic KL-divergence grid search when $0 < m - p \leq 3$, and s^2 itself for a saturated design. The MAP step re-runs the same gradient ascent from $\log \text{dispGeneEst}$ with the prior term enabled and, matching R, with the revert and the grid fallback disabled. An outlier rule then keeps the raw MLE wherever $\log \alpha_{\text{MLE}}$ exceeds the trend by more than $2s$.

The KL-divergence branch is the pipeline's only stochastic step. R estimates the prior variance by simulation: at each of 200 candidate variances x spanning $[0, 8]$ it draws 10^4 samples of $\log \chi^2_{m-p} + \mathcal{N}(0, \sqrt{x}) - \log(m - p)$, histograms them in half-unit bins against the

observed log-residual histogram to obtain a KL divergence, smooths the resulting 200-point curve with `loess` (local quadratic, span 0.2), and reports the argmin over a 1000-point grid, floored at 0.25. Two ingredients here are implementation-defined, and substituting either moves the answer.

The first is the draw. `set.seed(2)` fixes the stream inside R but not across implementations, so we replay R's stream itself: its LCG-based seeding of the Mersenne-Twister state (R does not use MT's standard initialization), its single-word conversion to a double, its inversion-based `rnorm` over Wichura's AS 241 quantile function, its Ahrens–Dieter `exp_rand`, and its `rgamma` (GS below shape 1, GD at or above). The last two are rejection samplers consuming a variable number of uniforms per draw, so the stream is inherently sequential and the loop is compiled with Numba rather than vectorized. One detail decides whether the replay stays in step: R's `rnorm` returns the mean without touching the generator when $\sigma = 0$, so the x=0 grid point draws no normal variates at all, and taking 10^4 of them there desynchronizes every later grid point.

The second is the smoother which builds a kd-tree over the predictor, fits value and slope only at the tree's vertices, and cubic-Hermite interpolates between them. We reproduce that construction, matching R's 31 median cuts to ∼5e-15 and its `predict` to ∼1e-14.

With both in place the branch reproduces R exactly: on `airway` (m=8, p=5) the simulated KL curve agrees to ∼5e-15 and the estimator returns R's prior variance, 0.5285285285285285, bit for bit. Getting either ingredient wrong is measurable: a different generator shifts the variance by 0.016 and a Savitzky–Golay smoother by 0.080, degrading final dispersion agreement from 0.045% to 1.4% and 6.7% respectively. The compiled replay adds ≈0.3 s of host time, once per dataset.

2.4 Three execution modes for the dispersion loop

The gradient-ascent loop is where the launch-bound overhead concentrates (Sec. 3.3), so it is the only stage with alternative execution backends. The three modes compute the same

mathematics and differ in how the loop is dispatched (Table 1). They are selected per call (`fit_dispersions(..., use_cuda_graph, use_triton)`) or through the `GPU_DESEQ_ACCEL` environment variable.

Table 1: The three execution modes for the dispersion loop.

Mode	Dispatch	Fidelity	Designs
eager	one kernel per tensor op	reference	any P
graph	replay a captured CUDA graph	bit-identical to eager	any P
Triton	one fused kernel per gene	R parity, not bit-identical	$P \in \{2, \ldots, 6\}$

Both accelerated modes fall back to eager on CPU; Triton also falls back for $P > 6$. Triton's agreement with eager is quantified in Sec. 3.2.

2.5 CUDA-graph chunked replay

In eager mode each gradient-ascent iteration launches a handful of tiny kernels over 1–10 MB arrays, so the loop is dominated by launch latency (Sec. 3.4). Capturing the loop as a CUDA graph amortizes that cost into a single replay, but two obstacles must be overcome.

The first is early exit. A fixed full-length capture would force every gene to the maximum iteration count, which wastes the typical 15–40-iteration case. To prevent this we capture a K=10-iteration graph that reads and writes a set of persistent state buffers in place, and replay it in a loop with a convergence check between chunks. A chunk always runs to its boundary, so a gene that converges partway through executes further iterations.

The second is cuSOLVER. The Cox–Reid term needs $\log\det(b)$ and $\mathrm{tr}(b^{-1}\,\mathrm{d}b)$ with $b = X^\top W X$, but `slogdet`/`solve` host-synchronize and cannot be captured. Since b is symmetric positive-definite for full-rank X, we replace them with an unrolled no-pivot LU (`_logdet_and_tr_binv_db`) returning both quantities from elementwise and indexing operations alone, matching the `linalg` path to ∼1e-15 for $P = 2 \ldots 6$. The eager path uses the same routine, which is what makes the two bit-identical.

Before capture the chunk is run three times on a side stream, warming the caching allocator and any lazy kernel initialization. Captured graphs are cached on G, S, P, the

chunk length K, whether a prior term is enabled, the dtype and the device. Only *whether* there is a prior is selected on the device, because the prior variance lives in the buffer set as a scalar tensor the replay reads, so one graph serves datasets whose prior variances differ.

2.6 Triton full-loop fusion

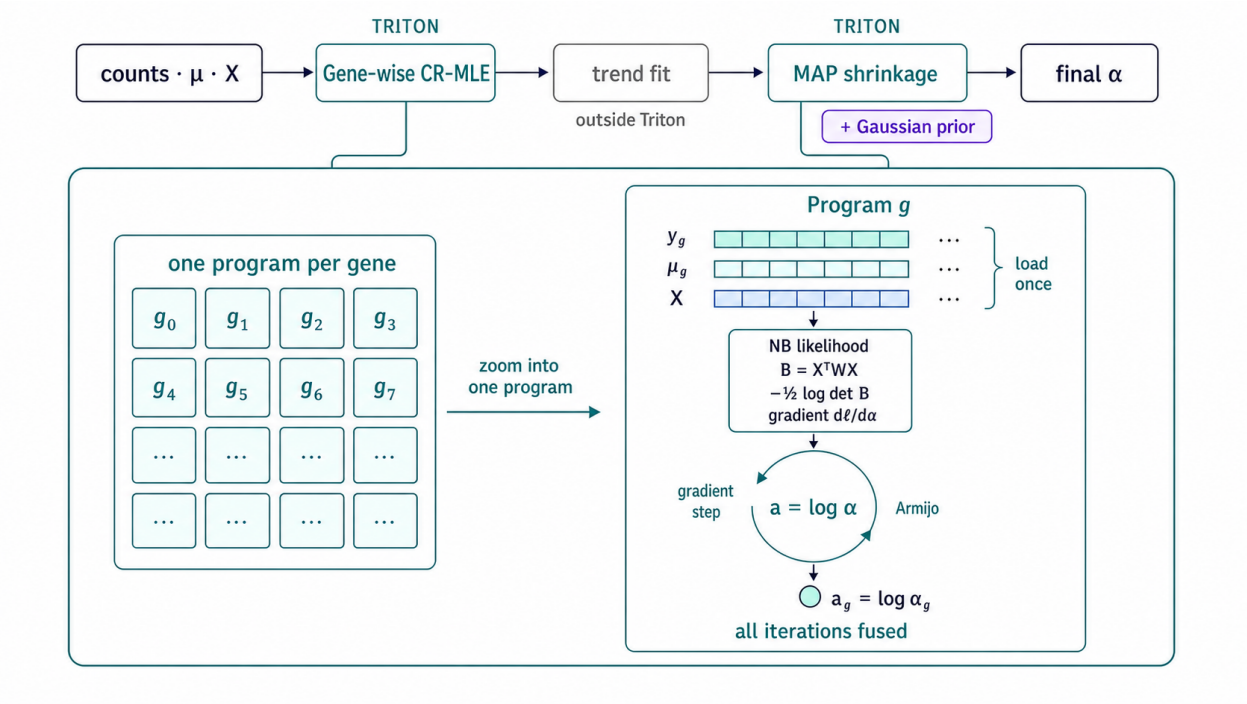

Figure 2: The fused Triton dispersion kernel. One kernel handles both gene-wise fits of the dispersion pipeline: the Cox–Reid MLE, and the MAP shrinkage with a Gaussian prior that follows the trend fit (the trend fit itself runs outside Triton). One Triton program is launched per gene; it loads $\mathbf{y}_g$, $\boldsymbol{\mu}_g$ and the design $\mathbf{X}$ once, then runs every iteration of the gradient ascent register-resident, with the NB log-likelihood, the Cox–Reid term $-\frac{1}{2}\log\det\mathbf{B}$ ($\mathbf{B} = \mathbf{X}^\top\mathbf{W}\mathbf{X}$), the $\log\alpha$-gradient and the Armijo line search all fused into the loop body. Only the converged $a_g = \log\alpha_g$ is written back to global memory. Supplementary Algorithms S1–S2 give the pseudocode.

The Triton mode fuses a gene's entire fit into a single program, launched once per gene (Fig. 2). Its counts, μ and design columns are loaded once into registers, and the gradient ascent then runs register-resident: the Cox–Reid $P \times P$ solve (closed-form 2×2 for P=2, an unrolled no-pivot LU above that), the NB log-likelihood and $\log\alpha$-gradient (`lgamma` and `log1p` from `libdevice`, `digamma` hand-implemented to match ATen's `calc_digamma` through

a recurrence to $x \geq 10$ followed by the Bernoulli asymptotic series), and the Armijo line search. A per-gene `while` loop lets each gene exit as soon as it converges, so the kernel does not run every gene to the batch maximum. The kernel is fp64 and covers $P \in \{2, \ldots, 6\}$; `fit_dispersions` gates on the covered range and routes wider designs to the eager path, while the kernel's own launcher raises instead of fitting a truncated model, since it reads a fixed number of design columns. At the gene-wise step, non-converged and high-dispersion genes get the same noIncrease revert and grid fallback as the eager path, computed eagerly after the kernel returns; at the MAP step neither applies, in either mode, because R applies neither. The fused reduction order and the hand-written `digamma` move the last bits, so this mode is not bit-identical to eager, and Sec. 3.2 reports how far the two diverge on real data.

Covering a range of P needed one extra step. The eager path writes the $P \times P$ elimination as a Python loop over tensor slices, but Triton's frontend admits no local arrays and no compile-time sequence building, so a register-resident matrix cannot be indexed by a loop variable: the algebra must appear as straight-line code specialized to each P. Because that grows quadratically, from 20 lines at P=2 to 174 at P=6, we emit it from a generator mirroring the eager elimination operation for operation rather than hand-writing each case; only the $P \in \{2, 4\}$ arms are hand-written. Both kinds of arm are checked against the eager Cox–Reid term at fixed α, which isolates the linear algebra from the optimizer's path: over all covered P and $S \in \{8, 12, 24\}$ the log-posterior agrees to 3e-15 relative and its gradient to 3e-14, against committed test bounds of 1e-13 and 1e-12.

2.7 Batched port of the apeGLM optimizer

apeGLM (Zhu et al., 2019) shrinks the log-fold-change by maximizing a per-gene posterior: the NB likelihood times a heavy-tailed Cauchy prior on the coefficient of interest and a wide Gaussian (SD 15, the DESeq2 default) on the nuisance coefficients. The Cauchy scale is set by empirical Bayes, $\tau = \min(\sqrt{\widehat{\mathrm{var}}}, 1)$, with $\widehat{\mathrm{var}}$ the root of apeGLM's prior-variance equation in the MLE coefficients and their Wald standard errors, solved once per dataset on

the host.

Reproducing the reference's output turns on a property of this objective. For extreme-effect, low-count genes it is bimodal in the shrunk coefficient, with a sharp likelihood mode far from zero and a prior mode near zero. The two modes can be nearly tied in posterior mass. R does not report the global optimum in that situation; it reports whichever mode its solver reaches from its initialization. DESeq2's `lfcShrink` runs apeGLM with `method="nbinomCR"`, whose estimate comes from a limited-memory BFGS solver (LBFGS++ via `RcppNumerical`) started at a fixed ± 0.1-alternating point. Which mode is reported is therefore a property of that solver and that starting point, not of the posterior alone, and no better-converging method recovers it.

We therefore port the reference's optimizer itself, vectorized across genes on the GPU: limited-memory BFGS with a 6-step history and a backtracking strong-Wolfe line search (ftol $= 10^{-4}$, wolfe $= 0.9$, dec$= 0.5$, inc$= 2.1$, at most 100 trials), the two-loop recursion, an initial step $1/\|d\|$ reset to 1 each iteration, apeGLM's gradient-norm and objective-change stopping tests, a cap of 300 iterations, and the identical ± 0.1 initialization. Running the same algorithm from the same start selects the same mode, reproducing R's shrunk log-fold-changes to the agreement reported in Sec. 3.2. The per-gene solver state (β_g, the 6-step history, step size, convergence flags) is carried as $(G, \cdot)$ tensors exactly as in the dispersion loop, so the whole shrinkage stage runs batched on the GPU with no serial per-gene fallback.

The reported `lfcSE` is the square root of the shrunk coefficient's diagonal of the inverse observed information at the optimum. Following `lfcShrink`, only the shrunk coefficient and its SE are replaced; the reported p-values remain the MLE Wald values.

2.8 Parity methodology

Reference fixtures are generated by `scripts/generate_r_fixtures.R` against R DESeq2 1.52.0: three single-factor designs at 12×200, 30×500 and 60×2000 (samples $\times$ genes, $P = 2$), a 60×1500 two-factor design $\sim$ batch + condition with a three-level batch

($P = 4$), and a 60×1000 continuous-covariate design ($P = 2$). Every intermediate is checked against R: size factors, `dispGeneEst`, `dispFit` (the parametric trend on every fixture, plus the `"local"` and `"mean"` trends on the single-factor ones), `dispMAP`, the final dispersion and its outlier set, β, μ, the hat-matrix diagonals, Cook's distances, Wald SE, the LRT statistic on all five designs, the independent-filtering `padj`, and the apeGLM prior scale and shrunk LFC/SE. Errors are reported as p95 relative values, or absolute values where a quantity crosses zero. Sec. 3.2 reports the measured agreement for the principal quantities and tabulates the five pipeline stages on the real datasets (Table 2).

3 Results

3.1 Experimental setup

All GPU measurements are taken on one NVIDIA A100-SXM4-40GB under PyTorch 2.6 with CUDA 12.4 and NVIDIA driver 550.90.07. The reference CPU baseline (host A) is R DESeq2 1.52.0 on a 12-core box, with R 4.6.0 and apeglm 1.34.0. Every timing is a warm median of N repetitions with `torch.cuda.synchronize()` on both sides of the timed region, so no GPU work is left in flight when the clock stops. R runs single-threaded in the primary comparisons.

We validate on three real, published RNA-seq datasets sliced into six designs spanning $P = 2$–5 model terms: pasilla (Brooks et al., 2011) (*Drosophila*, n=7, one- and two-factor), airway (Himes et al., 2014) (human, n=8, three design slices), and a 300-sample GTEx (GTEx Consortium, 2020) whole-blood vs. skeletal-muscle contrast (54,922 genes). The first two are distributed as Bioconductor experiment packages; the GTEx counts are recount2's uniform reprocessing of SRA study SRP012682 (Collado-Torres et al., 2017), from which we draw 150 samples per tissue at a fixed seed and convert base coverage to read counts by dividing by the average read length. Scaling studies use fixed-seed negative-binomial simulations where the gene and sample axes can be swept independently.

3.2 Numerical parity with R

Correctness is checked per substep against R on all six real datasets (Table 2). A separate synthetic step-parity suite (`tests/test_r_step_parity.py`, 76/76) confirms the same agreement at controlled sizes, and more tightly: size factors agree to <1e-10, gene-wise dispersion to ∼1e-9, and MAP dispersion, β, μ, the Hessian and Wald SEs to ∼1e-6 relative. A further suite (`tests/test_triton_dispersion.py`) pins the fused kernel against the eager path once per covered P, the axis along which its specialized arms (Sec. 2.6) could diverge without any other test noticing.

Table 2: Per-substep agreement with R DESeq2 1.52.0 across all six real datasets. Each dataset is scored independently by the substep's natural metric (n=6; the score is per-dataset, not a pooled statistic), and both the median and the least-favorable of the six are shown. The R reference here is the substep chain (`estimateSizeFactors` → `estimateDispersions` → `DESeq` → `results`). Because size factors and dispersions already exist, the `DESeq` call times the Wald fit and, where applicable, count-outlier replacement and refitting. Reproduce: `bench/bench.py` → `bench/results/reference_parity.json`.

Substep	Metric	Median	Worst
normalization	max rel. Δ	4.1e-15	5.7e-15
dispersion	p95 rel. Δ	8.5e-4	4.4e-3
glm_fit	p95 $\lvert\Delta\rvert$ (LFC)	2.4e-5	8.1e-5
significance	Jaccard@0.05	1.00000	0.99963
lfc_shrink	Pearson r	0.99999	0.99722

All 30 substep checks pass, and every one of the six dispersion comparisons is within 0.44% of R, the worst being `gtex_blood_muscle` at 4.4e-3. The small-degrees-of-freedom case is not the outlier one might expect. R enters the simulation-based prior-variance estimator only when the residual degrees of freedom satisfy $0 < m - p \leq 3$, so `airway` (n=8, P=5, dof 3) is the only one of the six datasets to take that branch at all; because the branch reproduces R's scalar bit for bit (Sec. 2.3), its dispersion agrees to 4.5e-4, its raw LFC to 3.0e-5, and its significant-gene set matches R's *exactly* (Jaccard 1.000). The apeGLM shrinkage matches R to Pearson $r \geq 0.997$ on every dataset, with p95 $\lvert\Delta\rvert$ ≤3.4e-3. The residual is confined to flat, near-degenerate genes on which two limited-memory-BFGS implementations break a tie

between equal-posterior optima differently; the shrunk LFC is a reported effect size and does not feed the significance calculation, so it changes no call. Across the 25 synthetic dispersion configurations of Sec. 3.4 and Sec. 3.5, the graph path reproduces eager exactly: gene-wise, MAP and final dispersion all agree to max $|\Delta| = 0$ (`benchmarks/results_a100*.json`). Triton's fused reduction order and hand-written `digamma` change the last bits; its dispersion agrees with eager to $\leq$3.2e-7 on the five small-n sets. The standard-pipeline `gtex` comparison includes count replacement and refitting in every mode (Table 3).

3.2.1 Parity in the standard DE plots

The residuals above are summary statistics, but a DESeq2 run is read through a small set of standard diagnostic plots, and parity is only useful if it survives into those. Fig. 3 redraws the three an analyst looks at first (the dispersion-estimate, MA and volcano plots) from both pipelines by identical code, on axis limits taken from R and reused for cuDESeq2 so the rows are compared rather than each rescaled to its own data. They are visually indistinguishable: the gene-wise MLE cloud, the fitted trend, the shrunken MAP band and the 141 genes the outlier rule exempts from shrinkage all land in the same places, and the same 3,993 genes are called at padj < 0.05.

Fig. 4 quantifies the same agreement across all six designs in the terms a DE analysis is used for. The two pipelines put the same genes at the top of the ranked list: at every N up to 2,000 and on every design they share $\geq$99.6% of the top N, and their called sets agree to a Jaccard index of $\geq$0.961 across the whole range $\alpha \in [10^{-3}, 0.2]$. At α=0.05, four of the six agree exactly and the other two reach 0.99963 and 0.99997. Ranking uses |Wald statistic| rather than the p-value because on `gtex` R reports $p = 0$ for hundreds of underflowed genes, so a p-value ranking would compare arbitrary tie-breaks among them.

The first is a μ floor in the Wald standard error. R forms the weights for its coefficient covariance from a μ already thresholded at `minmu` (`fitBeta.cpp`), whereas our IRLS returns μ unthresholded because Cook's distance requires it raw; omitting the floor when forming

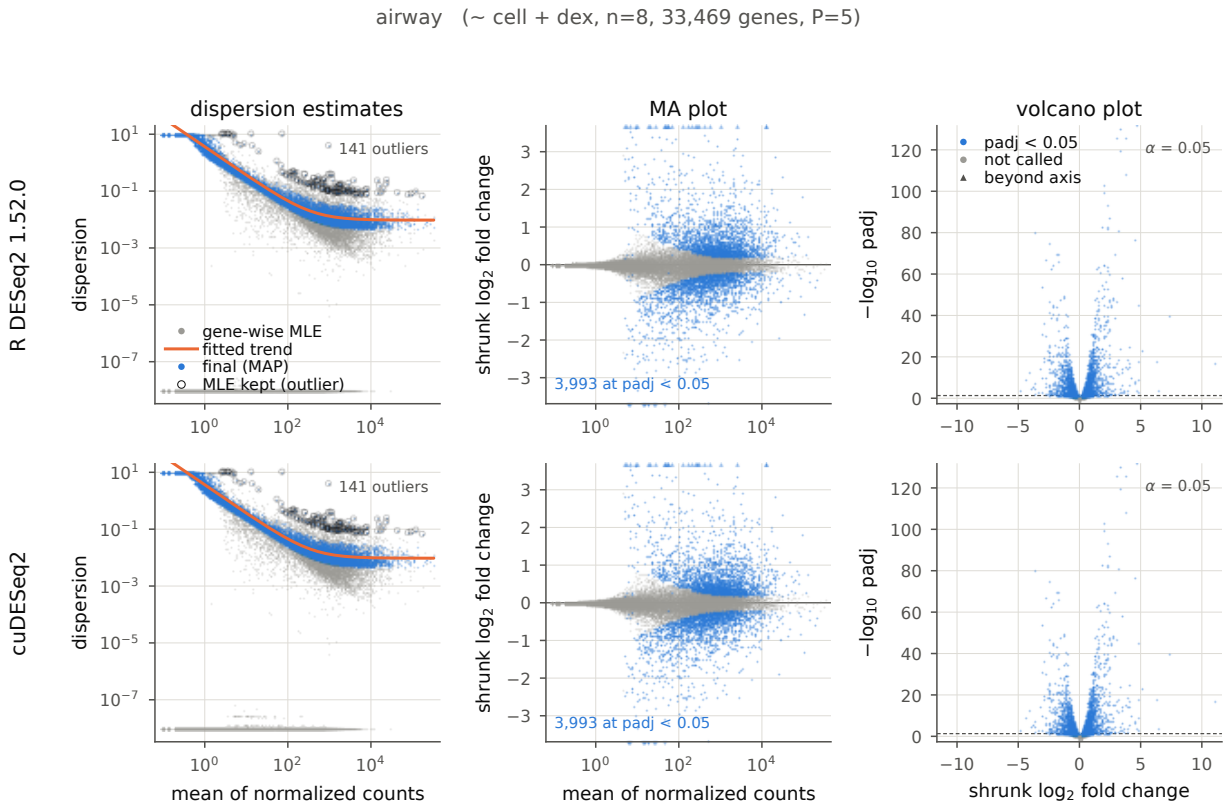

Figure 3: The three standard DESeq2 diagnostics on `airway` ($\sim$ cell + dex, n=8, P=5), drawn from R DESeq2 1.52.0 (top) and cuDESeq2 (bottom) by the same plotting code. *Left*, the `plotDispEsts` view: gene-wise MLE (grey), fitted trend (orange), final estimate (blue), and circled genes whose MLE exceeds the trend by more than two residual SDs and is therefore kept unshrunk. *Center*, MA plot of the apeGLM-shrunk LFC, with out-of-range genes on the boundary as in `plotMA`. *Right*, volcano plot. Axis limits are computed from R's output and reused for cuDESeq2. Reproduce: `validation/export_r_dispersion_details.R` then `validation/make_paper_figures.py`.

$X^\top W X$ for the SE drops any sample with $\mu < 0.5$ out of the weight matrix and inflates the SE. The effect is invisible on ordinary genes and severe on degenerate ones. Without the floor the SE runs 22% high on genes where one contrasted group is *entirely* zero, which is enough to change `airway_cell`'s called set and pull its Jaccard index against R's 206 genes to 0.971, and ∼7% high on near-zero-count genes whose dispersion is pinned at the ceiling, moving their raw p-values by up to 0.07. With the floor applied the SE matches R to 0.03% on those same degenerate genes, `airway_cell` and `airway` both reproduce R's called set *exactly*, and the histograms of panel (a) coincide. Only the few genes carrying $\mu < 0.5$ in some sample are affected at all, which is why every aggregate metric in Table 2 is essentially unchanged either way. A suite scoring only aggregates can miss a detail of this kind.

The standard wrapper has an additional branch when a design cell contains at least `minReplicatesForReplace`= 7 samples. Both implementations flag counts above the 0.99 quantile of the Cook's-distance F distribution, replace eligible counts with the 20%-trimmed mean of normalized counts rescaled by the sample size factor, and refit the affected genes. The refit re-estimates the gene-wise dispersion while retaining the original fitted trend and dispersion prior, then repeats the MAP and Wald fits. Original counts are retained for subsequent shrinkage. This branch is unreachable for the five seven- or eight-sample designs; it is exercised by `gtex`, with 150 samples in each group. Table 2 therefore tests the same standard wrapper path for both implementations rather than comparing cuDESeq2 with `nbinomWaldTest` alone.

3.3 Real-data timing versus R

The graph and Triton modes change the dispersion optimizer. The remaining stages use the same code, except that a standard-pipeline outlier refit invokes the selected dispersion optimizer again. None of the five small designs enters that branch. Their downstream timing differences are measurement noise or, for Triton, changes in the number of tolerance-based shrinkage iterations caused by last-bit dispersion differences.

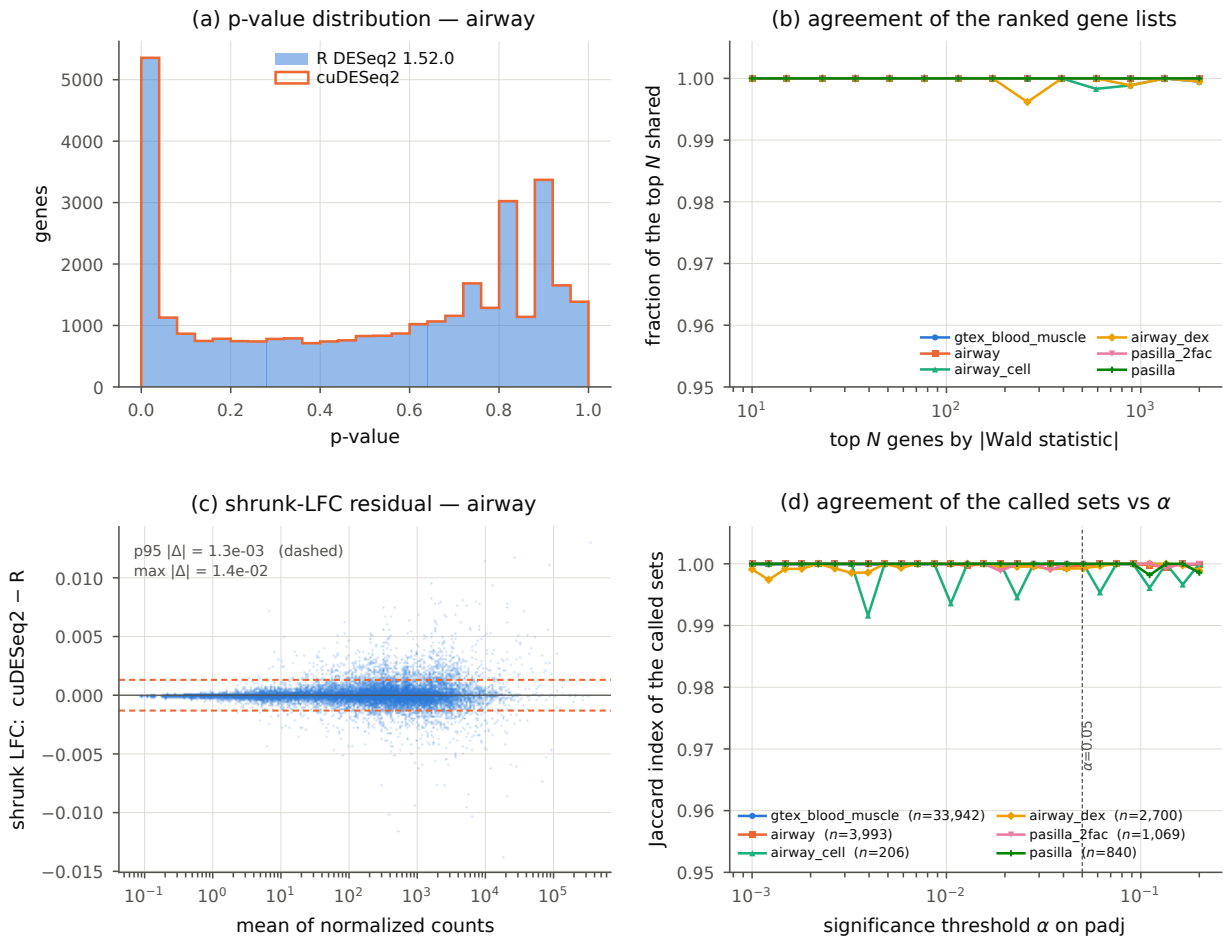

Figure 4: Quantitative agreement across all six designs. *(a)* p-value distributions on `airway`, overlaid. *(b)* Fraction of the top N genes shared by the two rankings, ranked by |Wald statistic| to avoid comparing tie-breaks among genes whose p-value has underflowed in R. *(c)* apeGLM-shrunk LFC residual against expression, showing no mean-dependent bias; dashed lines mark p95 $|\Delta|$. *(d)* Jaccard index of the called sets as the significance threshold moves, with the number of genes R calls at α=0.05 in the legend. The index is granular to $\sim 1/n$, so the small called sets move in visible steps. The `gtex` curve now includes count replacement and refitting in both pipelines.

Table 3: End-to-end wall time on the real datasets: single-threaded R 4.6.0/DESeq2 1.52.0 and cuDESeq2 on one A100. All six GPU rows are from the same standard-pipeline benchmark session; the `gtex` row includes count replacement and affected-gene refitting. Reproduce: `bench/run_r.R` and `bench/bench.py` → `bench/results/timings.json`.

			R DESeq2 (s)	cuDESeq2, one A100 (ms)		
Dataset	P	n	1 thread	eager	graph	Triton
pasilla	2	7	11.1	982	650	717
pasilla_2fac	3	7	12.1	1481	922	841
airway_dex	2	8	30.9	1307	968	898
airway_cell	4	8	36.1	4169	3579	3495
airway	5	8	41.6	2559	1681	1504
gtex	2	300	471.6	8510	8293	4754

The best-mode speedup over single-threaded R is 10.3–99.2×; `gtex` is the upper endpoint.

Triton is the fastest total mode on five designs; graph is fastest on `pasilla`. Triton has the fastest dispersion substep on all six: fusing the wider Cox–Reid solve (Sec. 2.6) cuts the dispersion substep by 8.9× on `pasilla_2fac` (P=3) and 2.7× on `airway` (P=5) against the eager loop (708→80 ms and 1586→581 ms, same run). `airway`'s dispersion substep also carries the ≈0.3 s CPU cost of the R-stream replay of Sec. 2.3, in every mode. All three modes are nonetheless shipped, because what separates them is coverage and fidelity rather than speed: graph is bit-identical to eager at any P, whereas Triton is a GPU-only path limited to $P \leq 6$ and agrees with eager only to the last few bits (Sec. 3.2).

3.4 Scaling on synthetic data

Synthetic data lets the gene and sample axes move independently, which is what identifies *why* the dispersion loop is slow rather than only how slow it is. We first establish the bound. Hardware counters were unavailable on this instance, so the roofline is measured empirically: a three-array fp64 streaming probe sustains 1369 GB/s, or 88% of the A100's 1555 GB/s specified peak, and the loop's kernels come nowhere near it. At $60 \times 20{,}000$ the elementwise weight update reaches 21% of sustained bandwidth and the `lgamma` and `digamma` evaluations 43% and 36%; at $60 \times 2{,}000$ all three fall to 3–8%, compute-bound on the special

Table 4: Per-substep wall time (ms) on the real datasets, single-threaded R on the reference host vs. the three cuDESeq2 modes. The `glm_fit` timing includes Cook's-distance replacement and refitting. Reproduce: `bench/bench.py` → `bench/results/TABLES.md`.

Dataset	Substep	R	eager	graph	Triton
pasilla	dispersion	1722	402	71	20
	glm_fit	3878	38	37	37
	significance	206	63	63	63
	lfc_shrink	5143	478	478	596
	total	**11091**	**982**	**650**	**717**
pasilla_2fac	dispersion	2140	708	161	80
	glm_fit	3755	320	321	322
	significance	185	74	60	60
	lfc_shrink	5912	378	379	378
	total	**12137**	**1481**	**922**	**841**
airway_dex	dispersion	4707	429	109	51
	glm_fit	9924	55	53	53
	significance	1397	145	124	123
	lfc_shrink	14715	676	681	670
	total	**30932**	**1307**	**968**	**898**
airway_cell	dispersion	5794	748	186	60
	glm_fit	9584	37	36	36
	significance	362	114	115	115
	lfc_shrink	20157	3268	3240	3283
	total	**36080**	**4169**	**3579**	**3495**
airway	dispersion	8077	1586	713	581
	glm_fit	12686	62	60	60
	significance	872	117	117	117
	lfc_shrink	19712	793	790	746
	total	**41570**	**2559**	**1681**	**1504**
gtex	dispersion	188957	3500	3623	200
	glm_fit	239691	3147	2810	2709
	significance	496	623	623	621
	lfc_shrink	39330	1235	1233	1220
	total	**471588**	**8510**	**8293**	**4754**

`normalization` omitted (≤4 ms cuDESeq2 on every set); full versioned R records are in `bench/results/timings.json`. All GPU values come from the same A100 session.

functions rather than starved for bandwidth. Neither regime is the binding constraint. What settles it is the assembled objective-and-gradient evaluation `_lp_and_dlp`, the body of the loop: 1287 µs at 20,000 genes and 1292 µs at 2,000. A tenfold change in arithmetic and bytes moved produces no change in wall time. This is the signature of a loop paying mainly for launches and dispatch. That is the cost graph replay and Triton fusion attack, and why they buy more than a better-tuned kernel could.

The synthetic scaling experiments below isolate the dispersion optimizer rather than report end-to-end DESeq2 speedups; they therefore do not exercise the standard replacement/refit wrapper branch. The real-data timings in Table 3 use that branch where applicable. If the loop is launch-bound, the accelerators should pay most where iterations are many and each is cheap, and fade as every iteration acquires enough arithmetic to fill the device. The sample axis tests that directly (Table 5). Holding genes at 2,000 and sweeping S from a 2v2 pilot to a 2,000-sample cohort, the graph's speedup on the isolated gene-wise ascent loop declines from 7.6× at S=4 to 0.96× at S=2000: by two thousand samples the launch overhead it removes is a rounding error against the per-iteration work, and replay is no longer worth its bookkeeping. Triton decays over the same range but never loses, from 69.0× to 7.7×, since fusion removes the intermediate traffic as well as the launches. Two features of the sweep are structural rather than noise. The S=4 and S=6 rows run the full 100-iteration cap while every larger S converges in 13–25, which is most of why the small-sample regime is both the slowest and the most accelerable. And S=4 is the one row where the stage-level speedup (1.8×) falls far below the loop-level one (69.0×): with $m - p = 2$ that design takes the stochastic KL branch (Sec. 2.3), whose compiled replay of R's random stream is serial host work no GPU mode touches, contributing ≈470 ms to every mode at S=4 and nothing at S=6. It is Amdahl's remainder for that row.

The gene axis says the same thing from the other side. Taking 2,000 to 20,000 genes at fixed S=60 multiplies the work tenfold but the eager stage time only by 1.7× at P=2 (127→210 ms) and 1.2× at P=4 (193→224 ms): nine tenths of the added genes are absorbed

Table 5: Sample-axis scaling at 2,000 genes, `~condition` (P=2), single A100. "GW loop" is the isolated gene-wise Cox–Reid gradient-ascent fit (Sec. 2.3), the part the accelerators act on; "full stage" is `fit_dispersions` around it, including host-side trend fitting. Speedups are against eager in the same row. The graph is bit-identical to eager at every S ($\max|\Delta| = 0$); Triton is not, and agrees to ≤3.0e-8 here. Reproduce: `benchmarks/bench_cuda_graph.py --sample-sweep` → `benchmarks/results_a100_samplesweep.json`.

		GW loop (ms)			GW speedup		full-stage speedup	
S	iters	eager	graph	Triton	graph	Triton	graph	Triton
4	100	350.2	46.1	5.1	7.59	68.99	1.61	1.81
6	100	335.2	46.7	5.1	7.18	66.05	6.20	40.35
30	13	44.9	10.8	3.1	4.15	14.29	3.81	13.70
60	15	51.5	11.4	3.2	4.51	16.11	3.50	14.10
200	16	55.3	14.7	3.2	3.77	17.43	3.10	13.92
1000	18	63.9	44.0	8.0	1.45	8.02	1.30	8.17
2000	25	128.3	133.5	16.6	0.96	7.71	1.08	6.88

into launch latency the small case already paid. Because that fixed overhead is amortized over more genes at 20,000, it is also where the graph has least left to remove, its stage speedup dropping from 4.4× to 1.7× at P=2 (3.7× to 2.2× at P=4). Graph capture is a warm-path optimization: the one-time capture costs 487–796 ms across these four configurations, so it is recovered only when many matrices of one shape are fitted in a session, as in the serving pattern the virtual-cell evaluation loops of the Introduction generate.

3.5 Ablations

Three design choices in Sec. 2.4–2.7 could reasonably have gone the other way, and each is worth separating from the headline numbers.

Chunk length, and why a full-length capture fails. The obvious way to graph an iterative loop is to capture all of it, and that is the worst configuration we measured. Because a captured graph cannot branch on a convergence flag, a capture of k iterations must be replayed $\lceil \text{iters}/k \rceil$ times, so the loop executes $\lceil \text{iters}/k \rceil \cdot k$ iterations rather than the iters eager early-exits after; done genes are frozen no-ops but still cost their share of the kernel. Sweeping k over the divisors of maxit = 100 (Table 6) traces that cost model.

The full-length capture k=100 rounds 26 real iterations up to 100 and gives back almost all the benefit: 1.83× instead of 6.01× at 2,000 genes, 0.99× instead of 2.42× at 20,000, where it is indistinguishable from not graphing at all. What the sweep shows is that the padded iteration count, not the replay count, sets the time: rows executing the same number of iterations land within a couple of per cent of each other however many replays that took (k=25 and k=50 both pad to 50 and differ by 0.4% at 2,000 genes; k=2 through k=20 all pad to 40 and differ by 2% at 20,000). At 2,000 genes k=2 is fastest (6.01×) because it avoids padding, whereas at 20,000 genes k=10 and k=20 tie at 2.42×. The current k=10 default is therefore a conservative compromise rather than a universal optimum.

Table 6: `GRAPH_CHUNK` sweep on the isolated gene-wise loop, single A100, `~condition`. "iters run" is $\lceil \text{iters}/k \rceil \cdot k$, the padded count the replay actually executes; the eager loop early-exits after 26 and 40 iterations respectively. k=100 captures the whole loop in one graph and is therefore the fixed-full-length-capture ablation. Every k is bit-identical to eager ($\max|\Delta| = 0$), so these are pure scheduling trade-offs. Reproduce: `benchmarks/bench_cuda_graph.py --sweep` → `benchmarks/results_a100_sweep.json`.

	60 × 2,000 (eager 89.2 ms)			60 × 20,000 (eager 138.8 ms)		
k	iters run	graph (ms)	speedup	iters run	graph (ms)	speedup
2	26	14.83	**6.01**	40	58.24	2.38
5	30	16.28	5.48	40	57.48	2.41
10	30	16.18	5.51	40	57.29	**2.42**
20	40	20.73	4.30	40	57.45	**2.42**
25	50	25.42	3.51	50	71.15	1.95
50	50	25.34	3.52	50	70.96	1.96
100	100	48.73	1.83	100	139.66	0.99

Per-gene early exit in the fused kernel. The Triton kernel's loop is guarded per program by `(it < maxit) & (~done)` (Sec. 2.6) rather than run for a fixed trip count, and the iteration counts of Table 5 bound what that is worth: outside the small-sample regime the loop converges in 13–25 iterations against a cap of 100, so a fixed-trip kernel would execute four to nearly eight times the loop body for an identical answer. The saving is largest exactly where the batch is widest, since a fixed trip count makes every gene wait for the cap rather than for its own convergence. This is a bound inferred from the shipped

kernel's measured iteration counts rather than a measured A/B: the released code exposes no fixed-trip switch to compare against.

The apeGLM optimizer is a parity choice, not a speed choice. A batched Newton solver for the shrinkage step is both simpler to write and faster than porting apeGLM's L-BFGS, and it disagrees with R. The reason is the one described in Sec. 2.7: the Cauchy-prior posterior is bimodal for extreme-effect genes, R reports whichever mode its own optimizer reaches from its own initialization, and a solver that finds the global optimum therefore returns a better estimate that is not the reference's. This is the one place where improving on the reference and reproducing it are in direct conflict, and a drop-in replacement has to choose reproduction. Porting the reference's optimizer restores agreement to $r \geq 0.997$ on shrunk log-fold-changes across all six designs (Table 2) while keeping the stage on the GPU with no per-gene host fallback.

4 Discussion

cuDESeq2 preserves the DESeq2 model and its numerical settings. It uses fp64, the reference convergence tolerances, and the same optimizer sequences, clamps, and line searches. Table 2 reports agreement for size factors, gene-wise and MAP dispersions, coefficients, Wald statistics, adjusted p-values, and apeGLM-shrunk effects. The measured end-to-end gain on the six A100 comparisons is 10.3–99.2×.

The dispersion loop is limited mainly by launch and dispatch latency at the tested sizes (Sec. 3.4). Its working set is small, it uses only a fraction of sustainable memory bandwidth, and runtime follows iteration count more closely than gene count. Batching replaces sequential per-gene fits with tensor operations. CUDA-graph replay and Triton fusion reduce the remaining per-iteration overhead. The benefit is largest for designs with few samples, many genes, and many optimizer iterations, and declines as each iteration contains enough work to occupy the device.

Numerical compatibility required details that are not specified by the statistical model. R's random-number stream and its kd-tree `loess` implementation affect the dispersion prior at small residual degrees of freedom (Sec. 2.3). The μ floor in `fitBeta.cpp` affects Wald standard errors for degenerate genes (Sec. 3.2.1). For extreme effects, apeGLM's L-BFGS initialization can select among local modes (Sec. 2.7). Matching the published equations without these implementation details yields valid estimates, but not the same output as DESeq2.

Speedup also depends on the CPU, worker count, and dominant pipeline stage. The current controlled comparison is single-threaded (Sec. 3.3); it should not be read as a hardware-independent ratio. Small designs are often shrinkage-bound, whereas the largest cohort is dominated by dispersion and outlier refitting.

4.1 Limitations

- Tested designs: single factor (up to four levels), additive multi-factor (up to P=5), continuous. Not yet: interaction designs. Parity is not tested below n=7, the smallest real design here; the synthetic fixtures start at 12 samples.
- Not implemented: `lfcShrink` types `normal`/`ashr`, `glmGamPoi`, VST/rlog.
- Triton path: $P \in \{2, \ldots, 6\}$ (falls back to eager otherwise); not bit-identical to eager; agreement is $\leq$3.2e-7 in dispersion on the five small-n sets, with a worst case of 2.6e-1 relative on one `gtex` gene.
- Benchmarks: a single GPU (one A100), so we cannot report how the three modes reorder on a smaller or newer device. Parity is measured against DESeq2 1.52.0.

Future work. The remaining compatibility work includes the `normal` and `ashr` shrinkage estimators, interaction and many-level factor designs, and extension of the fused kernel past P=6. Independent filtering and the parametric dispersion-trend fit remain on the host

because neither contains a per-gene optimizer to batch. They become larger fractions of runtime as the gene-wise fits get cheaper. The per-gene independence also permits sharding across devices, but multi-GPU execution has not been measured. The parity methodology here is specific to DESeq2, but the failure modes it caught (an implementation-defined random stream, a smoother that is not the estimator it approximates, a floor buried in a C++ inner loop) also apply to ports of other statistical software.

Funding

This work was supported by Synthesize Bio. It received no specific grant from any funding agency in the public, commercial or not-for-profit sectors.

Conflict of Interest

All authors are affiliated with Synthesize Bio, which developed and maintains the software described here and released it under the MIT license. The authors declare no other competing interests.

Data and software availability

Every dataset used here is public and previously published; this work generated no new sequencing data. pasilla (Brooks et al., 2011) and airway (Himes et al., 2014) are the Bioconductor experiment packages of the same names, installable with `BiocManager::install`. The GTEx cohort (GTEx Consortium, 2020) is taken from recount2 (Collado-Torres et al., 2017) as the gene-level summarized experiment for SRA study SRP012682, downloaded from the `recount-opendata` S3 bucket. The scripts `fetch_and_reference.R` and `prepare_gtex.R`, both under `validation/`, fetch each dataset, fix the contrast levels, and write the exact count matrices and R DESeq2 reference outputs the comparisons use, so all

six designs are reconstructible from the two scripts.

Source code, tests, and benchmark harnesses are available at `https://github.com/synthesizebio/gpu_deseq`, released under the MIT license. The package is distributed as `gpu-deseq` and imported as `gpu_deseq`; it requires Python 3, PyTorch and a CUDA GPU, with Triton for the fused mode. A Docker image definition pins R 4.6.0, Bioconductor 3.23, DESeq2 1.52.0, apeglm 1.34.0, PyTorch 2.6.0/CUDA 12.4 and the LaTeX toolchain. The same image runs CPU reference generation and, through the NVIDIA container runtime, the GPU benchmark. Reproduce with:

```
make container-build
make container-check
make container-test
make container-r-reference
make container-gpu-benchmark
make container-paper
```

References

Constantin Ahlmann-Eltze and Wolfgang Huber. glmGamPoi: fitting gamma–poisson generalized linear models on single cell count data. *Bioinformatics*, 36(24):5701–5702, 2020. doi: 10.1093/bioinformatics/btaa1009.

Angela N. Brooks, Li Yang, Michael O. Duff, Kasper D. Hansen, Jung W. Park, Sandrine Dudoit, Steven E. Brenner, and Brenton R. Graveley. Conservation of an RNA regulatory map between Drosophila and mammals. *Genome Research*, 21(2):193–202, 2011. doi: 10.1101/gr.108662.110.

Charlotte Bunne, Yusuf Roohani, Yanay Rosen, et al. How to build the virtual cell with artificial intelligence: priorities and opportunities. *Cell*, 187(25):7045–7063, 2024. doi: 10.1016/j.cell.2024.11.015.

Leonardo Collado-Torres, Abhinav Nellore, Kai Kammers, et al. Reproducible RNA-seq analysis using recount2. *Nature Biotechnology*, 35(4):319–321, 2017. doi: 10.1038/nbt.3838.

GTEx Consortium. The GTEx consortium atlas of genetic regulatory effects across human tissues. *Science*, 369(6509):1318–1330, 2020. doi: 10.1126/science.aaz1776.

Blanca E. Himes, Xiaofeng Jiang, Peter Wagner, Ruoxi Hu, Qiyu Wang, et al. RNA-Seq transcriptome profiling identifies CRISPLD2 as a glucocorticoid responsive gene that modulates cytokine function in airway smooth muscle cells. *PLoS ONE*, 9(6):e99625, 2014. doi: 10.1371/journal.pone.0099625.

Wenxing Hu, Haotian Zhang, Yu H. Sun, Shaolong Cao, Jake Gagnon, Yuka Moroishi, Yirui Chen, Zhengyu Ouyang, and Baohong Zhang. ScaleSC: a superfast and scalable single-cell RNA-seq data analysis pipeline powered by GPU. *Bioinformatics Advances*, 5(1):vbaf167, 2025. doi: 10.1093/bioadv/vbaf167.

Michael I. Love, Wolfgang Huber, and Simon Anders. Moderated estimation of fold change and dispersion for rna-seq data with DESeq2. *Genome Biology*, 15(12):550, 2014. doi: 10.1186/s13059-014-0550-8.

Boris Muzellec, Maria Teleńczuk, Vincent Cabeli, and Mathieu Andreux. PyDESeq2: a python package for bulk RNA-seq differential expression analysis. *Bioinformatics*, 39(9): btad547, 2023. doi: 10.1093/bioinformatics/btad547.

NVIDIA Corporation. CUDA C++ programming guide: CUDA graphs. `https://docs.nvidia.com/cuda/cuda-c-programming-guide/`, 2024. Accessed July 2026.

Adam Paszke et al. PyTorch: An imperative style, high-performance deep learning library. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2019.

Yusuf H. Roohani, Tony J. Hua, Po-Yuan Tung, et al. Virtual Cell Challenge: toward a Turing test for the virtual cell. *Cell*, 188(13):3370–3374, 2025. doi: 10.1016/j.cell.2025.06.008.

Philippe Tillet, H. T. Kung, and David Cox. Triton: an intermediate language and compiler for tiled neural network computations. In *Proc. 3rd ACM SIGPLAN Int. Workshop on Machine Learning and Programming Languages (MAPL)*, pages 10–19, 2019. doi: 10.1145/3315508.3329973.

Zhiting Wei, Yiheng Wang, Yicheng Gao, et al. Benchmarking algorithms for generalizable single-cell perturbation response prediction. *Nature Methods*, 23(2):451–464, 2026. doi: 10.1038/s41592-025-02980-0.

Anqi Zhu, Joseph G. Ibrahim, and Michael I. Love. Heavy-tailed prior distributions for sequence count data: removing the noise and preserving large differences. *Bioinformatics*, 35(12):2084–2092, 2019. doi: 10.1093/bioinformatics/bty895.

Supplementary algorithms

Supplementary Algorithm S1 gives a high-level view of the fused Triton dispersion fit sketched in Fig. 2; Supplementary Algorithm S2 gives the full kernel, including the Armijo line search, the step-size adaptation, and the Cox–Reid objective/gradient routine shared by the CR–MLE and MAP passes.

Supplementary Algorithm S1 Fused Triton dispersion fit (high-level)

Require: counts $\mathbf{Y} \in \mathbb{N}_0^{G\times S}$; fitted means $\boldsymbol{\mu} \in \mathbb{R}_+^{G\times S}$; design $\mathbf{X} \in \mathbb{R}^{S\times P}$; initial dispersions $\boldsymbol{\alpha}^{(0)}$; optional MAP prior.
Ensure: optimized log dispersions $\mathbf{a}$.

Launch one Triton program for every gene g in parallel.
  Load $\mathbf{y}_g$, $\boldsymbol{\mu}_g$, and $\mathbf{X}$ once into program-local state.
  Initialize $a \leftarrow \mathrm{clip}(\log \alpha_g^{(0)}, -30, 10)$ and step size $\kappa \leftarrow \kappa_0$.
  repeat inside the same fused program
    Compute the negative-binomial log-likelihood and Cox–Reid adjustment.
    If fitting MAP dispersion, add the Gaussian prior on $a = \log \alpha$.
    Compute the gradient $q \leftarrow \partial \ell / \partial a$.
    Propose the gradient-ascent step $\widetilde{a} \leftarrow a + \kappa q$.
    Accept or reject $\widetilde{a}$ using the Armijo condition; adapt κ.
  until converged or *maxit* is reached.
  Store the final a_g once in global memory.

Supplementary Algorithm S2 Detailed fused Triton dispersion kernel (CR–MLE or MAP)

Require: counts $\mathbf{Y} \in \mathbb{N}_0^{G\times S}$; fitted means $\boldsymbol{\mu} \in \mathbb{R}_+^{G\times S}$; design $\mathbf{X} \in \mathbb{R}^{S\times P}$, $P \in \{2,\dots,6\}$; initial dispersions $\boldsymbol{\alpha}^{(0)} \in \mathbb{R}_+^{G}$; compile-time $h \equiv$ HasPrior; prior means $\mathbf{m}$ and variance σ^2 when $h = 1$; initial step bound κ_0.
Ensure: optimized log dispersions $\mathbf{a}$ and iteration counts $\mathbf{c}$.

parallel for Triton program $g \in \{0,\dots,G-1\}$ **do** (one program per gene)
  Load $\mathbf{y}_g$, $\boldsymbol{\mu}_g$, and $\mathbf{X}$ once into program-local state.
  Set $a \leftarrow \mathrm{clip}\Big(\log \alpha_g^{(0)}, -30, 10\Big)$, $\kappa \leftarrow \kappa_0$, $n_{\mathrm{acc}} \leftarrow 0$, and $c \leftarrow 0$.
  Set $(\ell, q) \leftarrow \mathrm{ObjectiveGradient}(a, \mathbf{y}_g, \boldsymbol{\mu}_g, \mathbf{X}, h, m_g, \sigma^2)$.
  while $c <$ *maxit* and not converged **do**
    $c \leftarrow c + 1$ and propose $\widetilde{a} \leftarrow a + \kappa q$.
    If $q \neq 0$ and $\widetilde{a} \notin [-30, 10]$, shorten κ so that $\widetilde{a}$ lies on the violated boundary.
    Set $(\widetilde{\ell}, \_) \leftarrow \mathrm{ObjectiveGradient}(\widetilde{a}, \mathbf{y}_g, \boldsymbol{\mu}_g, \mathbf{X}, h, m_g, \sigma^2)$.
    Set $r \leftarrow [\,\widetilde{\ell} \geq \ell + \varepsilon\kappa q^2\,]$. (Armijo acceptance flag)
    Select $a' \leftarrow r\,?\,\widetilde{a} : a$, $\ell' \leftarrow r\,?\,\widetilde{\ell} : \ell$, and $\Delta \leftarrow \ell' - \ell$.
    Set $(\_, q') \leftarrow \mathrm{ObjectiveGradient}(a', \mathbf{y}_g, \boldsymbol{\mu}_g, \mathbf{X}, h, m_g, \sigma^2)$ and $q \leftarrow r\,?\,q' : q$.
    Set $n'_{\mathrm{acc}} \leftarrow n_{\mathrm{acc}} + r$ and $\kappa_{\mathrm{acc}} \leftarrow \min(1.1\kappa, \kappa_0)$.
    If r and $n'_{\mathrm{acc}} \bmod 5 = 0$, set $\kappa_{\mathrm{acc}} \leftarrow \kappa_{\mathrm{acc}}/2$.
    Set $\kappa \leftarrow r\,?\,\kappa_{\mathrm{acc}} : \kappa/2$, $n_{\mathrm{acc}} \leftarrow n'_{\mathrm{acc}}$, $a \leftarrow a'$, and $\ell \leftarrow \ell'$.
    Converge if $r \wedge (\Delta < 10^{-6} \ \vee \ a < \log(\mathit{minDisp}/10))$.
  end while
  Store $a_g \leftarrow a$ and $c_g \leftarrow c$ in global memory.
end parallel for
function ObjectiveGradient$(a, \mathbf{y}, \boldsymbol{\mu}, \mathbf{X}, h, m, \sigma^2)$
  Set $\alpha \leftarrow e^a$, $\mathbf{W} \leftarrow \mathrm{diag}(\boldsymbol{\mu}/(1 + \alpha\boldsymbol{\mu}))$, and $\mathbf{B} \leftarrow \mathbf{X}^{\mathsf{T}}\mathbf{W}\mathbf{X}$.
  Compute $\ell \leftarrow \ell_{\mathrm{NB}}(\mathbf{y}; \boldsymbol{\mu}, \alpha) - \frac{1}{2}\log\det(\mathbf{B})$.
  Compute $q \leftarrow \partial\ell/\partial a$, including $\mathrm{tr}\big(\mathbf{B}^{-1}\partial\mathbf{B}/\partial\alpha\big)$ via an unrolled 2×2 formula (P=2) or $P\times P$ LU solve.
  if $h = 1$ **then** $\ell \leftarrow \ell - (a - m)^2/(2\sigma^2)$ and $q \leftarrow q - (a - m)/\sigma^2$. (MAP only)
  return (ℓ, q).
end function